

QOrd-Net - Plain-Language Code Walkthrough

A spoken-word explanation of every file and every function in the project, plus how to run it and where the results appear. Written to be read aloud.

0. Read this first

This document explains the code as it stands today. One thing to keep in mind for your meeting: the code has moved on from the older written plan in the file CLAUDE.md. In the code today, the quantum circuit is trainable, meaning it learns during training. It uses eight qubits. And the experiment is simply the quantum branch switched on versus switched off. The older plan in CLAUDE.md described a fixed circuit that does not learn, and a five-part experiment. That is not what the code does now. The two files docs/notes.md and docs/results.md match the current code, so trust those.

1. The big picture - what the model is for

The project looks at photos of a young apple plant, taken on many different days with a special camera, and works out two things. First: is the plant infected with the disease called fire blight, yes or no. Second: how bad the disease is, on a scale from zero to three. It also makes a forecast: how bad the disease will be a few days after the last photo.

The camera is a hyperspectral camera. A normal photo has three colour channels, red, green and blue. This camera records two hundred and four channels, covering colours the eye cannot see. Each image is five hundred and twelve pixels wide, five hundred and twelve tall, and two hundred and four channels deep.

Severity is treated like a dial, not like four separate boxes. The model outputs two numbers, call them u and v . Picture them as a single point on a clock face. The angle of that point is the dial reading. Level zero sits at one angle, level one a further eighth of a turn round, level two another eighth, and level three another eighth. Because the levels are arranged in order around the dial, guessing level two when the truth is level one is only a small error, while guessing level three is a bigger error. Four separate boxes could not capture that idea of nearness.

The data has no official severity score, so the project builds one. For each day, it measures what fraction of the leaf area looks diseased. It takes the highest fraction seen so far, because a disease does not heal backwards in these experiments. Then it turns that fraction into a level from zero to three using three cut-off values.

2. How the data flows through the model, step by step

- Step one. Read the camera file for each day. Keep only three of the two hundred and four channels, to keep the work light.
- Step two. Line up the images. The plant shifts a little between days. Nudge every day's image so it sits on top of the first day's image.
- Step three. Run each image through DINOv2. This is a large image-understanding network, already trained by others, which the project does not change. It turns each image into a grid of sixteen by sixteen patches. Each patch is described by three hundred and eighty-four numbers.
- Step four. Find the diseased areas. Using a browning measure and an automatic threshold, mark which patches look diseased. Also count how many pixels are plant, which is needed for the area fraction.
- Step five. Build a small graph for each day. Each blob of connected diseased patches becomes a dot, called a node. Each node carries the average of its patch descriptions plus eight shape numbers, such as size, position and how round it is. Each node is joined to its four nearest neighbours. If there is no

disease that day, the code still makes one empty node, so the graph is never truly empty.

- Step six. A graph network called GATv2, run twice. Each node looks at its neighbours and updates its own numbers. Then the day is summarised by taking both the average and the maximum across all nodes, and squashing the result to a list of five hundred and twelve numbers.
- Step seven. A sequence network called a GRU, which is a standard memory network. It reads the daily summaries in order, and is also told how many days passed since the previous photo. It produces one summary of the whole time series, again five hundred and twelve numbers.
- Step eight. Two branches read that summary. The classical branch is a simple layer that produces thirty-two numbers. The quantum branch squeezes the summary down to eight numbers, feeds them into a small quantum circuit, and reads eight numbers back out. If the quantum branch is switched off, those eight numbers are simply zeros.
- Step nine. Fuse. Glue the thirty-two classical numbers and the eight quantum numbers together, and squash them to sixteen numbers.
- Step ten. Refinement loop. If the model is unsure of its answer, it nudges its own answer and tries again, up to five times.
- Step eleven. The heads, which are the final answers. The disease head says infected or not. The severity dial head gives the u and v for the dial. The progression step fits a straight line through the dial readings over time and reads that line a few days into the future.

3. The experiment, also called the ablation

Run the whole model twice with the exact same random seed. Once with the quantum branch switched on, once with it switched off. Everything else is identical. Any difference in the scores is therefore due to the quantum branch and nothing else.

Repeat this for many seeds. Report the average score and its spread, the average difference per plant, a confidence range worked out by resampling the numbers, which is called the bootstrap, and an effect-size number.

The current finding, written up in docs/results.md, is that there is no measurable difference. At this data size the quantum branch neither helps nor hurts.

4. How to run it

```
pip install -r requirements.txt
```

Install the packages the project needs.

```
python -m pytest tests.py
```

Run the automatic checks. Fast, and needs no dataset.

```
python train.py --seed 0
```

A quick dry run of the whole pipeline on fake random data.

```
python data.py
```

Run this once. It fixes which plants are training, validation and test, and it builds the cache of features. This is the slow step, because it reads the large camera files and runs DINOv2 on every image. It takes around twenty-five minutes.

```
python train.py --seed 0 --march
```

One real training run on the real data.

```
python train.py --seed 0 --march --no-quantum
```

The same, but with the quantum branch switched off.

```
python train.py --ablation --seeds 12
```

The full experiment. Trains twelve times with the quantum branch on and twelve times with it off, then compares.

5. Where the results show up

- On screen. Each run prints one line with the validation and test scores.
- runs, then the run name, then the seed number, then metrics.json. This holds the full numbers for that run, plus the training loss for every epoch.
- runs, then ablation. The same kind of files, one set for each on run and each off run in the experiment.
- runs, then ablation_summary.json. The overall comparison. It has the averages, the on minus off difference, the ninety-five percent confidence range, and the effect size.
- docs, then results.md. A short write-up in plain words of the latest experiment.
- plant_split.json. Which plants are in training, validation and test. Written once so it never drifts between runs.
- cache, then a short code, then the feature files. This is not a result, but it is where the slow preprocessing lands. The short code changes if you change any setting that affects preprocessing, so stale features are never reused by mistake.

6. The scores the project reports

- MAE, which is mean absolute error. The average size of the error in dial levels. Lower is better.
- Quadratic weighted kappa. An agreement score for ordered levels that punishes far-off mistakes more than near ones. One is perfect, zero is chance level.
- Disease accuracy. The fraction of infected or healthy calls that are correct.
- Progression MAE. The same as MAE, but for the forecast.
- Refinement passes. How many times the refinement loop ran. Tracked because it adds variation between runs.

7. File by file, function by function

config.py - the settings

- CONFIG. One big list of settings. Which channels to use, the severity cut-offs, the sizes of the layers, whether the quantum branch is on, the number of qubits, the number of training passes, the learning rate, and so on.
- config, with overrides. Hands back a copy of CONFIG with a few values changed, so one run can adjust settings without touching the shared original.

data.py - reading and preparing the data

Near the top of the file are fixed choices: eight shape numbers per node, four neighbours per node, a patch counts as diseased when at least a quarter of it is, the name of the DINOv2 model, and, for each of the two photo campaigns, March and July, which plant numbers are infected and which are healthy controls.

- read_hdr, given a path. The camera writes a small text header file next to each image, describing its shape and layout. This reads that text and pulls out the numbers the code needs.
- acquisition_date, given a header. Turns the date text in the header into a real calendar date. This is a helper. The main code actually uses the day number from the folder name instead.
- read_cube, given a path. Reads one full camera file into memory as a block of numbers, five hundred and twelve by five hundred and twelve by two hundred and four, and arranges it the right way round.
- read_bands, given a path and a list of channel numbers. Reads the file but keeps only the chosen channels.
- Timepoint. A small labelled bundle holding one day's information: the day number, the file path, and the diseased-pixel count for that day.
- _pos_count, given a folder. For infected plants the data includes a list of diseased pixel positions.

This counts how many there are. That count rising over time is the disease signal.

- `scan_campaign`, given a campaign name. Walks through all the day folders and plant folders for that campaign, and builds a list, for each plant, of its Timepoints in day order.
- `scan_index`. Does that for both campaigns and merges the two into one.
- `plant_status`, given a plant key. Says whether that plant is infected or a control, based on its number.
- `severity_eval_ok`, given a plant key. Returns false for the one infected plant whose lesions were never located, March plant five, so it is left out of severity scoring.
- `running_max`, given a list. Walks the list keeping the highest value seen so far. Missing entries simply repeat the last value. This encodes the idea that disease does not go backwards.
- `levels_from_fracs`, given fractions and cut-offs. Applies `running_max`, then turns each fraction into a level from zero to three by counting how many cut-offs it passed.
- `make_split`, given a seed. Decides which plants go to training, which to validation, and which to test. It balances infected against healthy, and March against July. The split is by whole plant, never by single day, because two days of the same plant look almost identical and would leak answers from test into training.
- `write_split`, given a seed. Works out that split and saves it to `plant_split.json`, so it is fixed from then on.
- `load_split`. Reads that file back.
- `align_frames`, given the stack of images. The alignment step. It finds how far each day's image has shifted compared with the first day, and slides it back into place.
- `DinoV2Encoder`. A wrapper around the DINOv2 image network. Its setup loads the pretrained network, freezes it so it never changes during training, and stores the standard image normalisation numbers. Its `encode` method resizes the images, normalises them, runs them through DINOv2, and returns the grid of patch descriptions, dropping the network's single summary token.
- `masks_and_area`, given the images and the grid size. For each day, works out which pixels are plant and counts them, computes a browning measure, uses an automatic threshold to decide which patches are diseased, and returns that sixteen by sixteen diseased-or-not grid together with the plant-pixel count.
- `frame_graph`, given the patch descriptions and the diseased grid. Turns one day's grid into a small graph. Each connected blob becomes a node, carrying the average patch description of its cells plus eight shape numbers. Nodes link to their nearest neighbours. If nothing is diseased, it returns a single empty node, so there is always a valid graph.
- `cache_key`, given the settings. Makes a short code from the settings that affect preprocessing. If any of them change, the code changes, and old cached files are not reused by accident.
- `cache_dir`, given the settings. The folder that matches that code.
- `build_cache`, given the settings. The slow one-time job. For every plant: read the images, align them, find the diseased grid, run DINOv2, build the daily graphs, and save all of that to disk so that training can simply load it.
- `_fake_graph`. Makes a random pretend graph, used for dry runs.
- `SyntheticSequences`. A fake dataset of random graphs with random labels, so the whole pipeline can be tested without the real data.
- `CachedSequences`. The real dataset. For each plant it loads the saved features, works out the area fraction for each day, turns that into levels, works out the gap in days between photos, and hands back everything the model needs.
- `collate`, given a batch. The model trains on several plants at once, but they have different numbers of days. This pads them all to the same length and records which days are real and which are padding.
- `build_dataset`, given the settings and which split. Returns the fake dataset or the real one, depending on the settings.
- The bottom of the file. Running `python data.py` saves the split and builds the cache.

model.py - the model itself

- `QuantumCircuit`. The quantum branch. Its setup creates a simulated eight-qubit quantum computer and defines the circuit. The circuit takes eight input numbers, turns them into rotations of the qubits, applies a trainable entangling layer, and repeats this three times, re-feeding the inputs each time. Re-feeding is a known trick for getting more out of a small circuit. It then measures each qubit, giving eight numbers between minus one and one. The trainable part is a small block of angles the optimiser can adjust, seventy-two numbers in total. Its forward method runs that circuit and returns the eight measured numbers.
- `_fit_line`, given points y and times t . Fits a straight line to some points. Used to project the dial forward in time.
- `QOrdNet`. The whole model. Its setup builds every piece: the two graph layers, the layer that squashes a day to five hundred and twelve numbers, the GRU, the classical branch, the small layer that feeds the quantum branch, the quantum branch itself, the fusing layer, the refinement layer, and the three answer heads.
- `backbone`, inside `QOrdNet`. Runs the graph layers on each day, summarises each day, runs the GRU over the days along with the day gaps, and returns one summary of the whole sequence, plus the per-day vectors, which are kept for the forecast.
- `fuse_features`, inside `QOrdNet`, given the summary. Runs the classical branch and, if it is switched on, the quantum branch. If it is off, the quantum part is zeros. Glues the two together and squashes to sixteen numbers.
- `refine`, inside `QOrdNet`, given the current answer. The try-again-if-unsure loop. It measures how unsure the answer is. If that is over the threshold, it nudges the answer and repeats, up to five times.
- `dial_traj`, inside `QOrdNet`. Produces the u and v dial reading for each day, which is needed for the forecast.
- `progression`, inside `QOrdNet`. Fits a line through the daily dial readings against elapsed time, and reads that line a few days past the last photo. This is the forward forecast. It has no ground truth to check against, so it is a demonstrated ability rather than a scored one.
- `forward`, inside `QOrdNet`, given a batch. Ties it all together and returns the disease answer, the severity dial, the per-day dials, and the forecast.
- `dial_loss`, given u and v and the true level. The training penalty for the dial. It is small when the predicted angle matches the true level's angle, and large when it does not. It is written a careful way so the maths stays smooth for training.
- `compute_loss`. Adds up the disease penalty, the severity penalty, and a smaller per-day penalty into one number to train on.
- `dial_to_level`, given u and v . Turns a dial reading back into a level from zero to three by reading its angle.
- `predicted_levels`, given the model output. Reads the level out of whichever severity head is in use.
- `confusion`, given predictions and truths. Builds a small table of predicted level against true level.
- `qwk`, given predictions and truths. Computes the quadratic weighted kappa score described earlier.
- `mae_levels`, given predictions and truths. The average size of the level error.

train.py - training, testing and the experiment

- `set_seed`, given a seed. Pins every source of randomness, so a run can be reproduced exactly.
- `loader`. Wraps a dataset so the trainer can pull batches from it.
- `train_one_epoch`. One pass over the training data. For each batch, make a prediction, measure the error, and adjust the model.
- `evaluate`, given the model and some data. Runs the model on held-out data without learning, and works out all the scores: severity error, kappa, disease accuracy, and a forecast check. The forecast check hides the last day, predicts it from the earlier days, and compares.

- run, given the settings and a seed. One complete run. Set the seed, load the data, build the model, train for the set number of passes, score on validation and test, save the numbers to a file, and print a summary line.
- mean_sd, given values. The average and the spread.
- bootstrap_ci, given values. A confidence range worked out by resampling the numbers many times, with replacement, and looking at the middle ninety-five percent. Used instead of a classic statistical test because the sample is small.
- cohens_d, given the differences. An effect-size number: the average difference divided by its spread.
- ablation, given the seeds and the number of epochs. The experiment. For each seed, train once with the quantum branch on and once with it off, everything else identical. Collect the scores, work out the average per-plant difference, its confidence range, and the effect size. Save it all to runs, then ablation_summary.json, and print a table.
- main. Reads the command-line options and either runs the experiment or a single run.

tests.py - automatic checks, run with `python -m pytest tests.py`

- Circuit checks. The quantum circuit has exactly one trainable block, of the right shape. Its output is eight numbers between minus one and one. Training signals flow back into it. And using more repeats really does change what it computes.
- Model checks. The model's output is the right size, with no broken numbers. When the quantum branch is off, it receives no training signal. That is the check that makes the experiment fair. On versus off really do produce different output. And one training step runs cleanly.
- Label checks. The running-highest logic and the fraction-to-level logic behave as expected, including when there is a gap in the photos after the disease has started.
- Graph checks. A day with no disease gives a single empty node. And a node's description really is the average of its patches.
- Score checks. Kappa is one for a perfect match, and it punishes far mistakes more than near ones. The angle-to-level conversion gives the right answers.
- Data check. If the real data folder is present, the folder scan finds the expected plants and days. Otherwise this check is skipped.

8. The open next steps, from docs/notes.md

- The forecast test is currently too easy, because severity flattens out and simply repeating the last value already does well. It should test the rising part of the disease instead.
- Only one test plant ever reaches the top severity level, so the range of severities is thin.
- Consider telling the model which campaign a plant is from, since March and July differ in variety, in how the plants were infected, and in camera distance.
- The May orchard photos could be used as a separate, real-world test.
- Leave-one-plant-out testing would give tighter confidence ranges.